



Università Politecnica delle Marche

A.A. 2015/2016

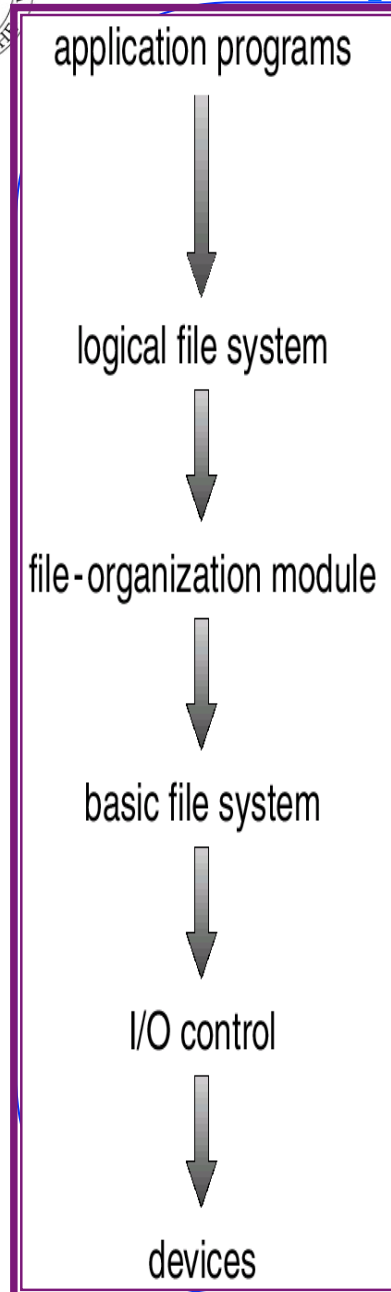
Gestione dell'I/O

Luca Spalazzi

spalazzi@dii.univpm.it

www.univpm.it/luca.spalazzi

File System Stratificato



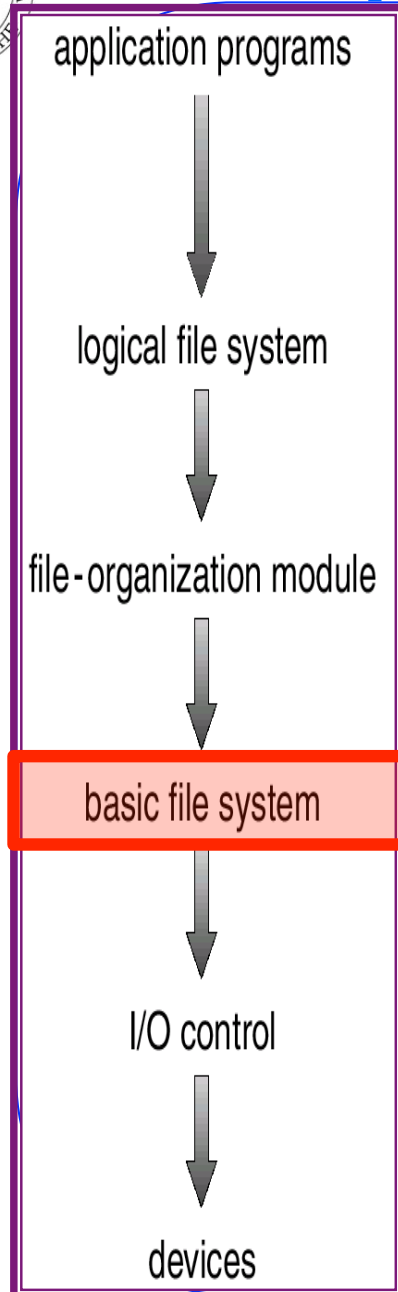
1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi



Gestione dell' I/O

- Gestione dell' Unità a Disco
 - Struttura del disco
 - Scheduling del disco
 - Gestione del disco
 - Strutture RAID
- Gestione dei dispositivi di I/O
 - Driver dei dispositivi di I/O
 - Hardware e Controller dei dispositivi di I/O
- Ciclo di vita di una richiesta di I/O
- Gestione dell' I/O in Linux
 - Struttura
 - Tipi di dispositivi
 - Scheduling del disco

File System Stratificato



1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi



Basic File System

- ✓ **Caching** – memoria veloce per tenere una copia dei dati che si stanno utilizzando
 - La cache contiene sempre una copia, il buffer non è detto
- ✓ **Buffering** – memorizzazione transitoria dei dati in memoria mentre essi sono trasferiti fra due dispositivi o tra una applicazione ed un dispositivo.
- **Spooling** – è una coda contenente dati per un dispositivo (per es. la stampante) che non può accettare flussi di dati intercalati
 - I dati provenienti da una singola applicazione si registrano in un specifico file su disco, quando il flusso è terminato, si inserisce il file nello spool
- **Prenotazione dei dispositivi** - sistema di prenotazione ed accodamento dei processi che richiedono l'uso di dispositivi non condivisibili
- **Gestione degli errori** – dovuti a motivi contingenti o permanenti. Gli errori spesso vengono riportati in un error log
 - Errori contingenti: il SO ritenta
 - Errori permanenti: viene restituito un codice di errore
- **Scheduling** - stabilire un ordine d'esecuzione efficace ad un insieme di richieste di I/O. Si applicano criteri simili a quelli visti per la CPU

Basic File System

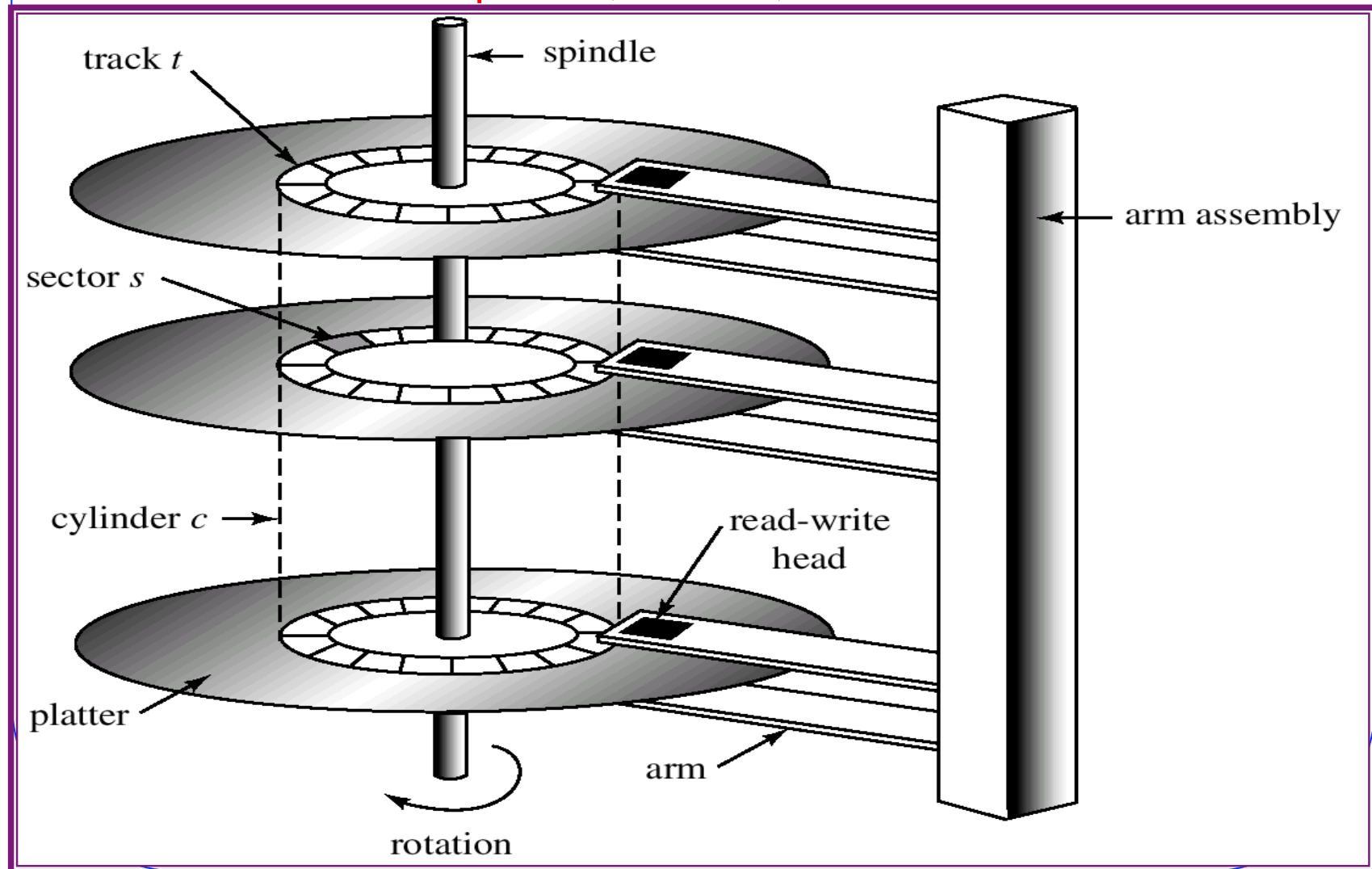
Gestione degli errori

- **Errori di programmazione:** il programmatore chiede qualcosa di impossibile/inconsistente (scrivere su un CD-ROM, tentare una seek su una seriale, accedere ad un dispositivo non installato, etc.)
 - È il sottosistema di I/O che si accorge dell'errore.
 - Azione: abortire la chiamata, segnalando l'errore al chiamante.
- **Errori del dispositivo.** Dipende dal dispositivo
 - È il controller che si accorge dell'errore, inviando una interruzione HW
 - Se transitori: cercare di ripetere le operazioni fino a che l'errore viene superato (rete congestionata)
 - Abortire la chiamata: adatto per situazioni non interattive, o per errori non recuperabili. Importante la diagnostica.
 - Far intervenire l'utente/operatore: adatto per situazioni riparabili da intervento esterno (es.: manca la carta).

Struttura del Disco

- A basso livello, le unità a disco sono suddivise in settori individuati dalla seguente tripletta

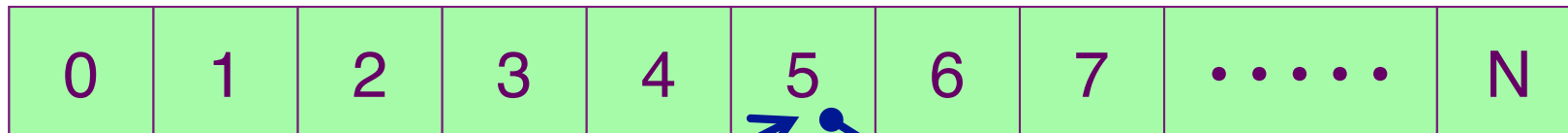
< superficie, cilindro, settore >.



Struttura del Disco

- Le unità a disco sono indirizzate come un unico grande array mono-dimensionale di blocchi logici, dove i blocchi logici sono l'unità di trasferimento più piccola.
- I **blocchi logici** dell'array mono-dimensionale vengono **associati** ai **settori** del disco in modo sequenziale.

Indirizzo blocchi del disco: I



$$I = (n * C * S) + (c * S) + s$$

$$s = I \bmod S$$

$$c = (I \div S) \bmod C$$

$$n = (I \div S) \div C$$

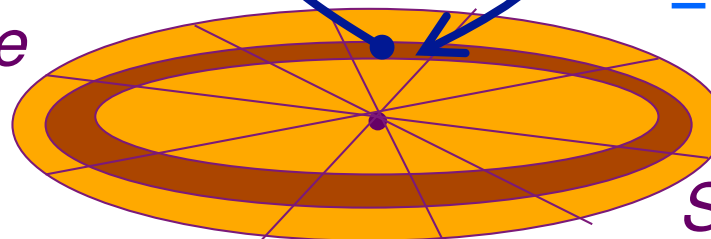
$$= I \div C * S$$

n : superficie

c : cilindro

s : settore

Indirizzo fisico: <n,c,s>



S = settori/cilindro

C = cilindri/superficie



Scheduling del Disco

Garantire un uso efficiente dell'unità a disco significa avere tempi di accesso contenuti e ampiezza di banda elevata.

■ **Tempo di accesso** è composto principalmente da:

- **Tempo di ricerca (seek time)**: il tempo necessario alle testine per posizionarsi nel cilindro contenente il settore desiderato.
- **Latenza di rotazione**: il tempo necessario affinché il disco ruoti fino a quando il settore desiderato si trova sotto la testina.

Minimizzare il tempo di accesso di fatto significa minimizzare il tempo di ricerca (è difficile minimizzare il tempo di latenza in quanto le moderne unità a disco non rivelano la posizione fisica dei blocchi logici).

Per minimizzare il tempo di ricerca si considera

tempo di ricerca $\approx$ distanza di ricerca

■ **L'ampiezza di banda** è il numero di byte trasferiti diviso il tempo intercorso fra la prima richiesta e il completamento dell'ultimo trasferimento.



Scheduling del Disco

- I diversi algoritmi di scheduling vengono illustrati prendendo in considerazione i cilindri che contengono i blocchi richiesti. Questo è dovuto alla scelta di considerare solo il tempo di ricerca.

- L'esempio di riferimento è il seguente:

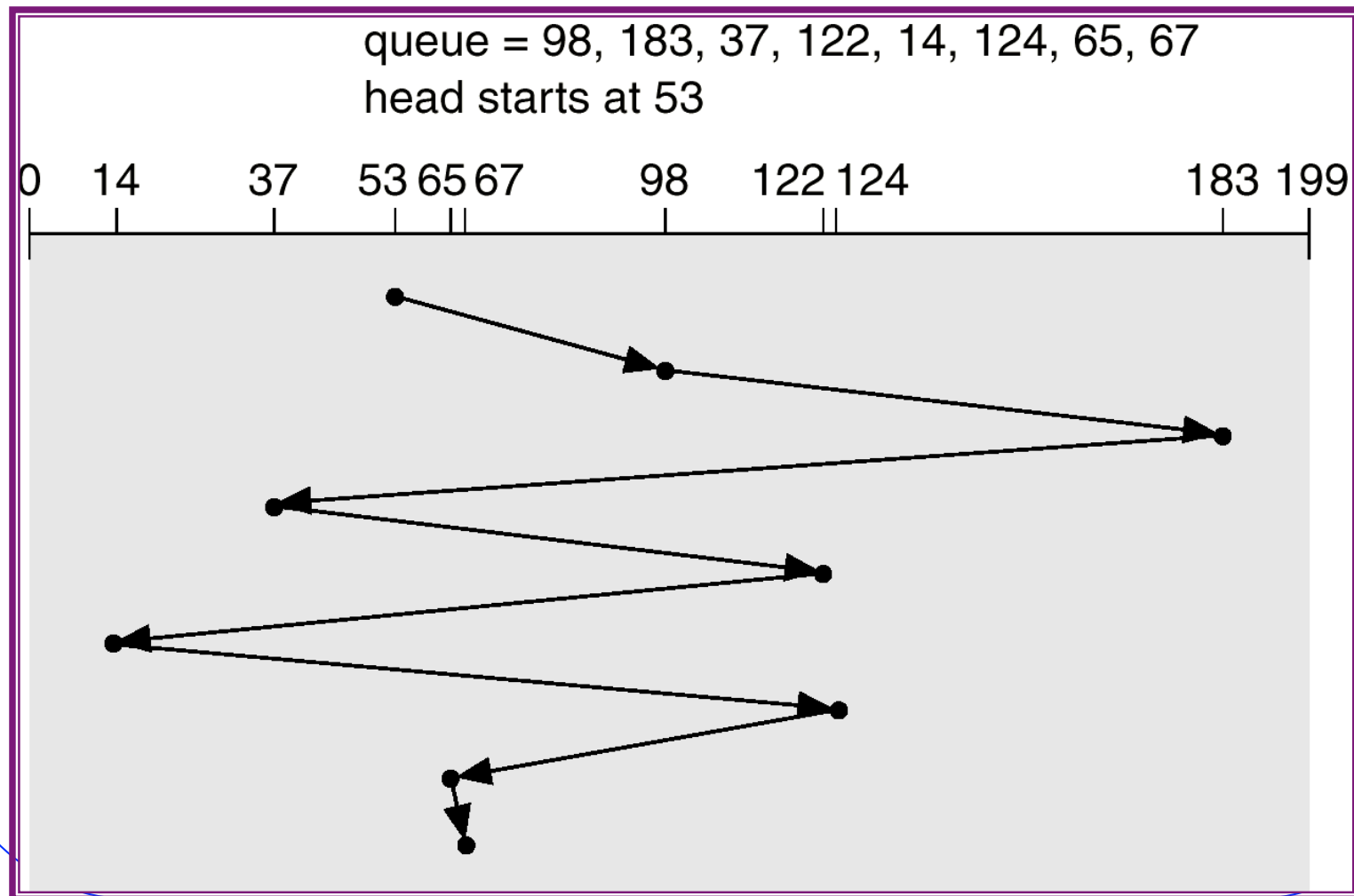
- supponiamo di avere la seguente coda di richieste (esprese dal numero di cilindro corrispondente)

98, 183, 37, 122, 14, 124, 65, 67

- supponiamo che la testina si trovi inizialmente al cilindro 53

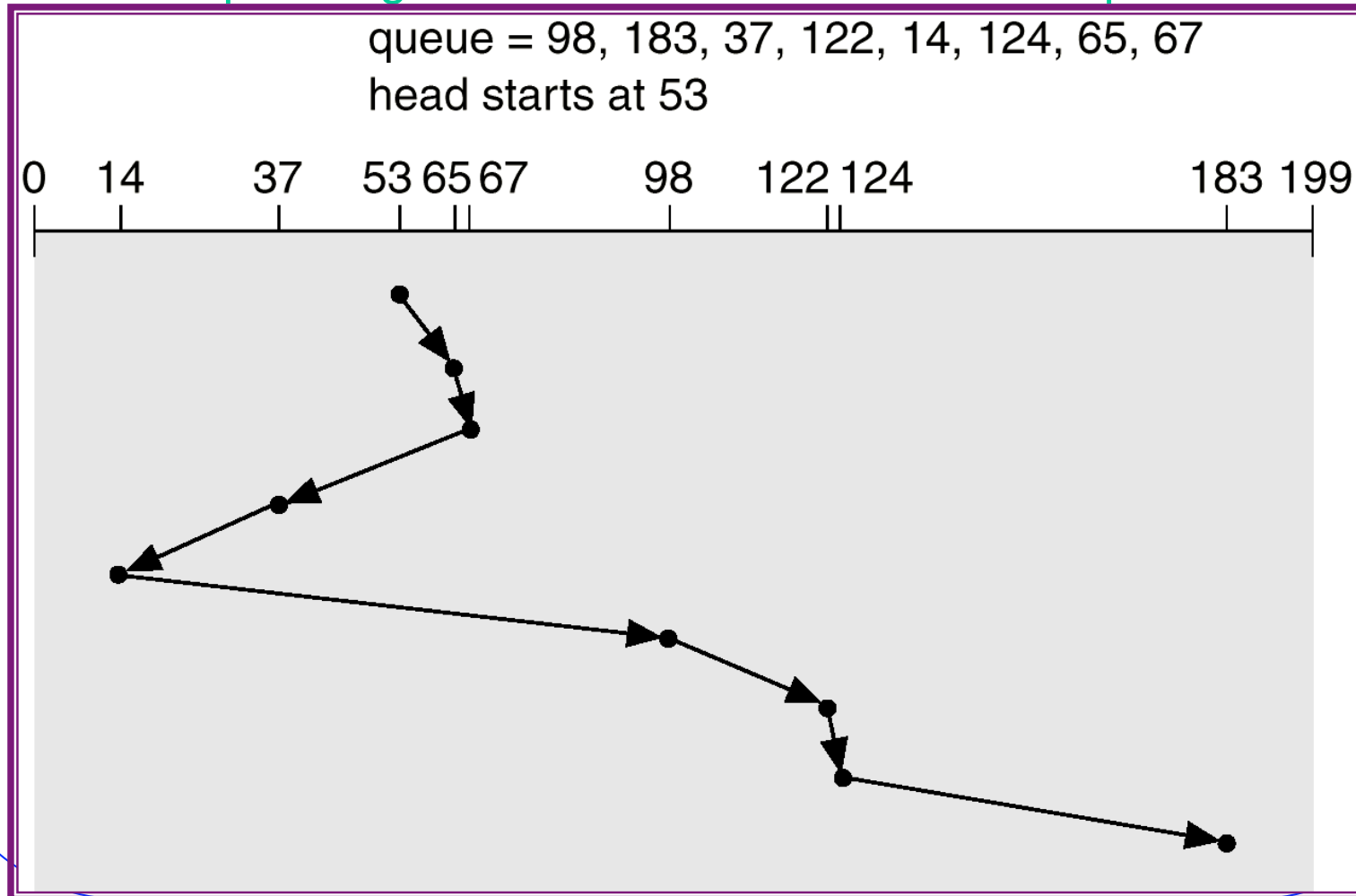
First Come First Served (FCFS)

- Seleziona la richiesta secondo l'ordine di arrivo.
- Nell'esempio in figura la testina si muove in totale per 640 cilindri



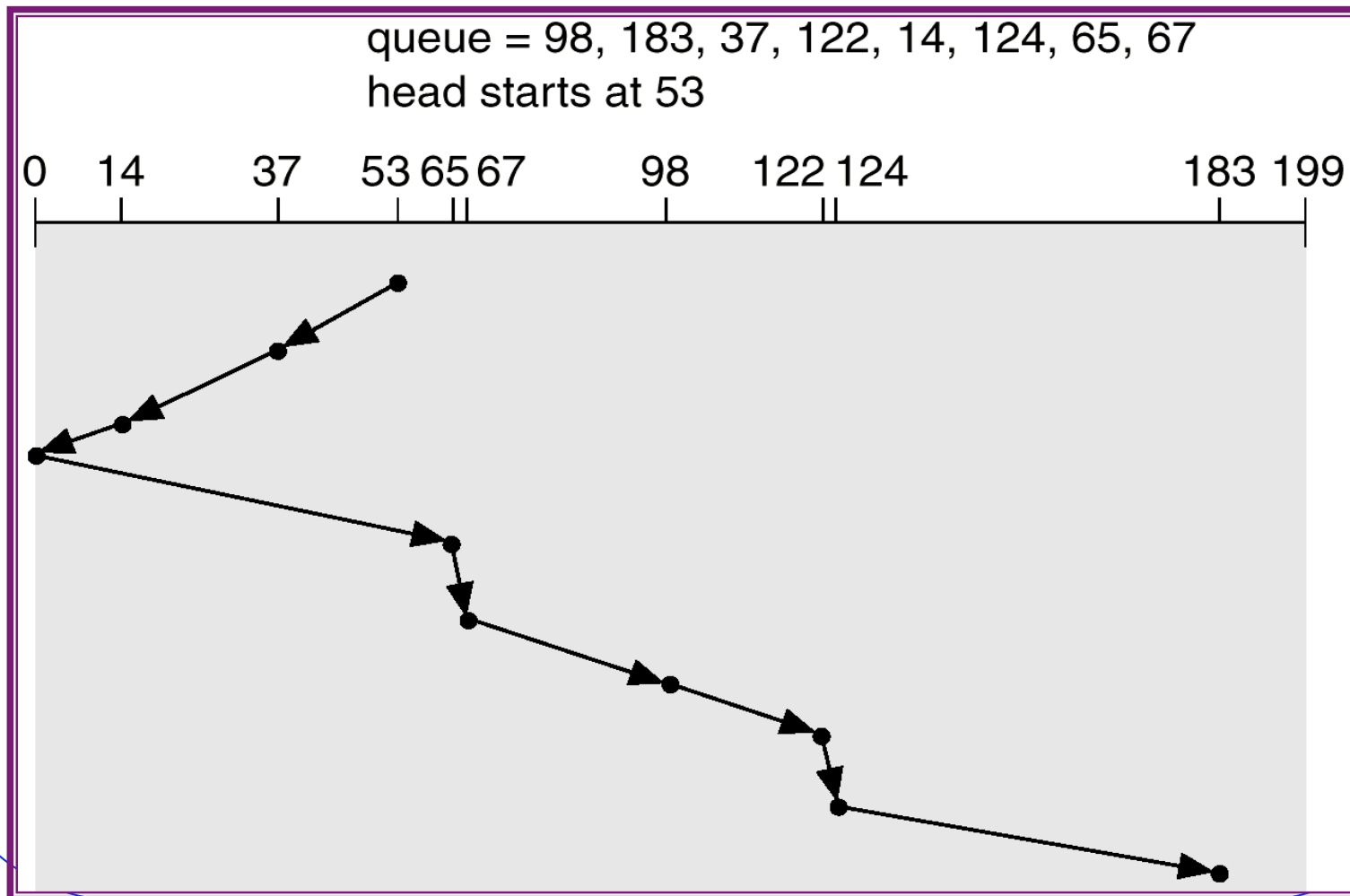
Shortest Seek Time First (SSTF)

- Seleziona la richiesta con il minimo tempo di ricerca rispetto alla posizione corrente della testina.
- SSTF è una specie di SJF; può causare starvation.
- Nell'esempio in figura la testina si muove in totale per 236 cilindri



Scheduling per Scansione (SCAN)

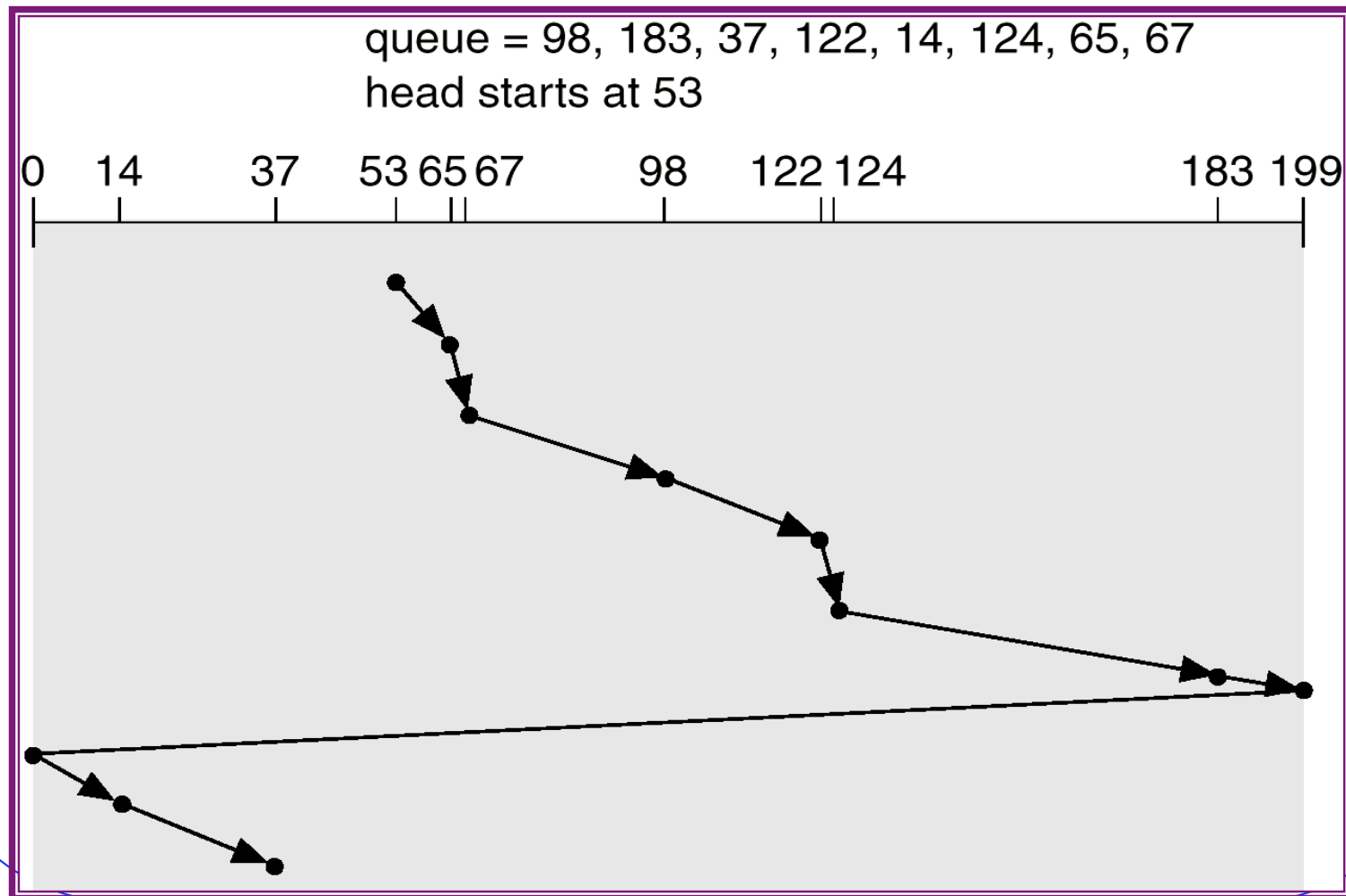
- Il braccio del disco parte da un estremo del disco e si muove verso l'altro estremo per poi tornare indietro, servendo le richieste mano a mano che le incontra (**algoritmo dell'ascensore**).
- Nell'esempio in figura la testina si muove in totale per 208 cilindri





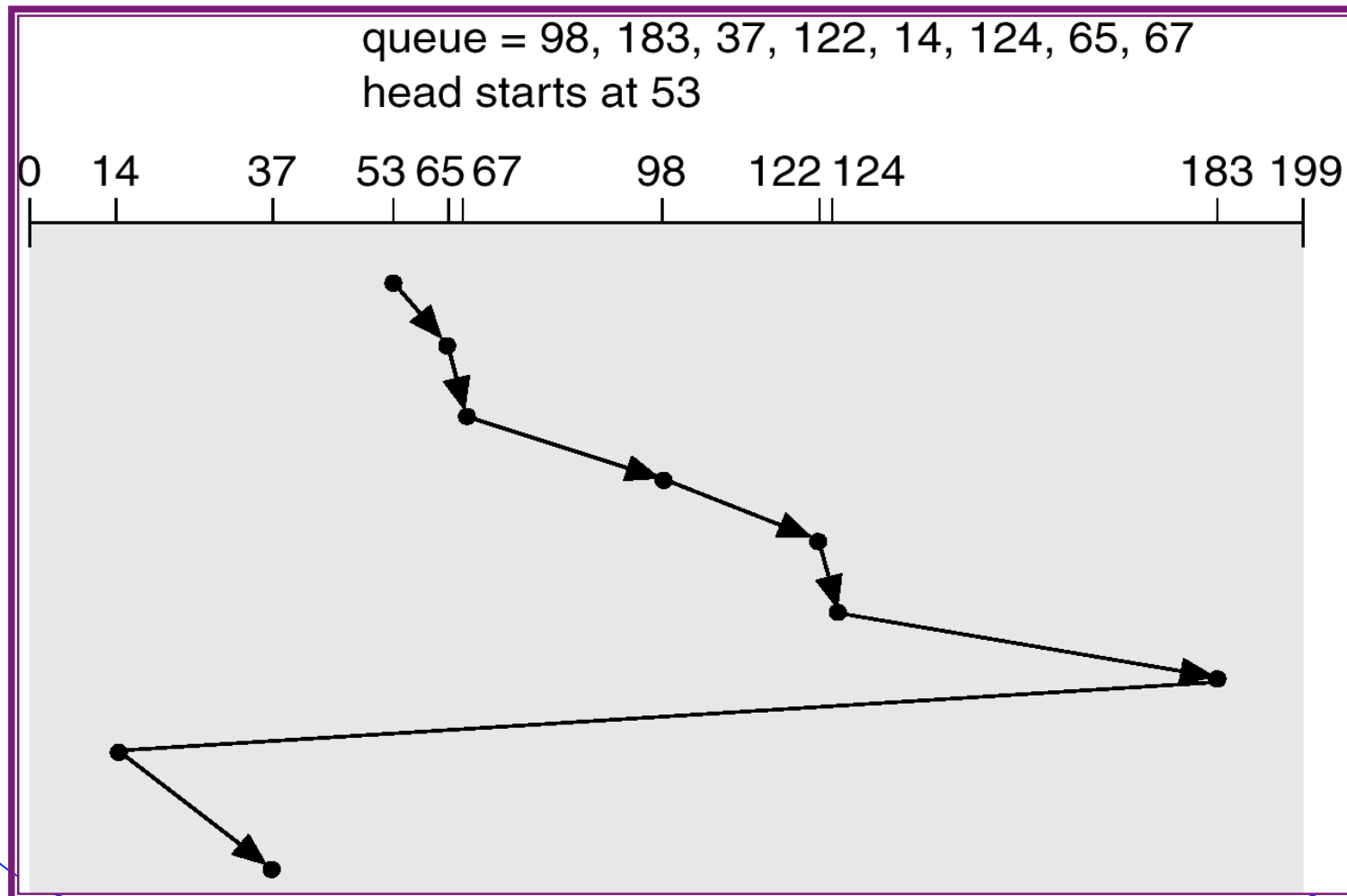
Scheduling per Scansione Circolare (C-SCAN)

- Come SCAN, solo che arrivato ad un estremo del disco, riparte verso l'altro senza servire richieste durante il viaggio di ritorno.
- Tratta i cilindri come una lista circolare.



LOOK e C-LOOK

- Varianti di SCAN e C-SCAN. Il braccio si sposta verso una estremità solo fino a quando ci sono altre richieste da servire in quella direzione.
- In figura è riportato un esempio di C-LOOK.



Scheduling del Disco

Table 11.2 Comparison of Disk Scheduling Algorithms

(a) FIFO (starting at track 100)		(b) SSTF (starting at track 100)		(c) SCAN (starting at track 100, in the direction of increasing track number)		(d) C-SCAN (starting at track 100, in the direction of increasing track number)	
Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed
55	45	90	10	150	50	150	50
58	3	58	32	160	10	160	10
39	19	55	3	184	24	184	24
18	21	39	16	90	94	18	166
90	72	38	1	58	32	38	20
160	70	18	20	55	3	39	1
150	10	150	132	39	16	55	16
38	112	160	10	38	1	58	3
184	146	184	24	18	20	90	32
Average seek length	55.3	Average seek length	27.5	Average seek length	27.8	Average seek length	35.8



Gestione dell' Unità a Disco

- **Formattazione a basso livello, o formattazione fisica** —
Suddivisione del disco in settori che il controller del disco può leggere e scrivere.
- **Partizioni** — Suddivisione del disco in uno o più gruppi di cilindri. Ogni partizione viene gestita dal SO come una unità disco a se stante.
- **Formattazione logica** — Creazione del File System.
- **Blocco d'avviamento (boot block)** — Contiene il programma di bootstrap (la ROM contiene il *bootstrap loader*).
- **Area di swap** — È l' area del disco utilizzata dal SO per la memoria virtuale. Può essere collocata:
 - all' interno del normale file system
 - in una partizione a se stante

Strutture RAID

(Redundant Array of Inexpensive Disks)

I dischi RAID sono batterie ridondanti di dischi

■ Miglioramento delle prestazioni tramite il parallelismo:

- *Sezionamento dei dati (data striping)* – consiste nel distribuire i bit di ciascun byte su più dischi.

■ Miglioramento dell'affidabilità tramite la ridondanza:

- *Copiatura speculare (mirroring o shadowing)* - ogni disco logico consiste di due dischi fisici e ogni scrittura si effettua su entrambi i dischi. Se uno dei dischi si guasta, si possono utilizzare i dati dell'altro disco.
- *Sezionamento + bit di parità (o + codici di correzione degli errori)* – per diminuire il grado di ridondanza richiesto si può pensare di combinare il sezionamento con l'uso dei codici di correzione degli errori (o con l'uso dei bit di parità). Questi codici associano ad ogni byte uno o più bit il valore dei quali dipende dai valori dei bit contenuti nel byte stesso.



(a) RAID 0: non-redundant striping



(b) RAID 1: mirrored disks



(c) RAID 2: memory-style error-correcting codes



(d) RAID 3: bit-interleaved Parity



(e) RAID 4: block-interleaved parity

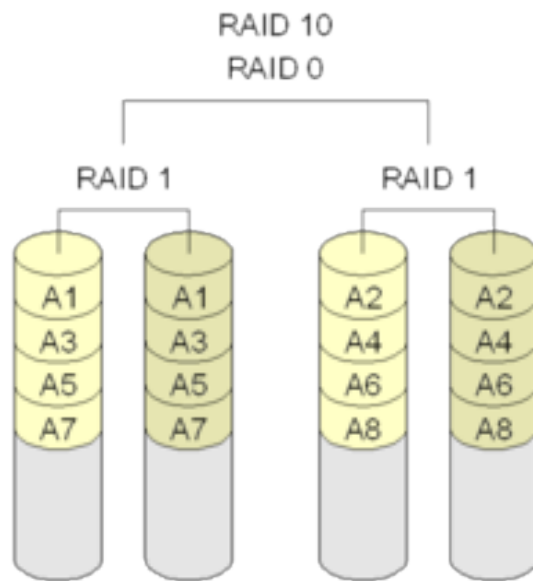
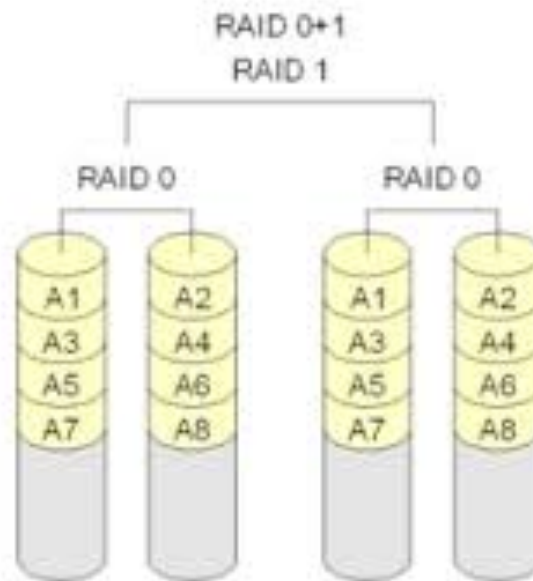


(f) RAID 5: block-Interleaved distributed parity

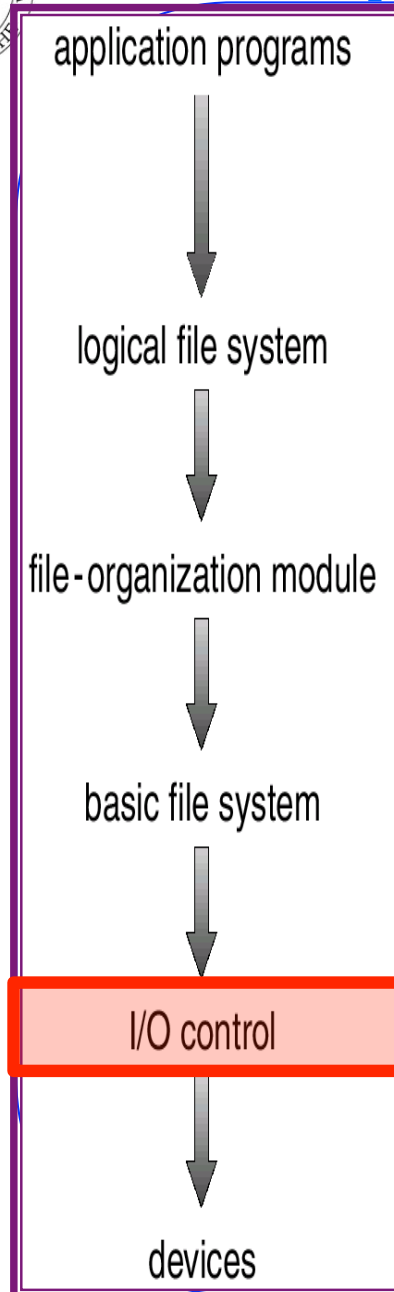


(g) RAID 6: P + Q redundancy

Livelli RAID (0 + 1) e (1 + 0)



File System Stratificato



1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi

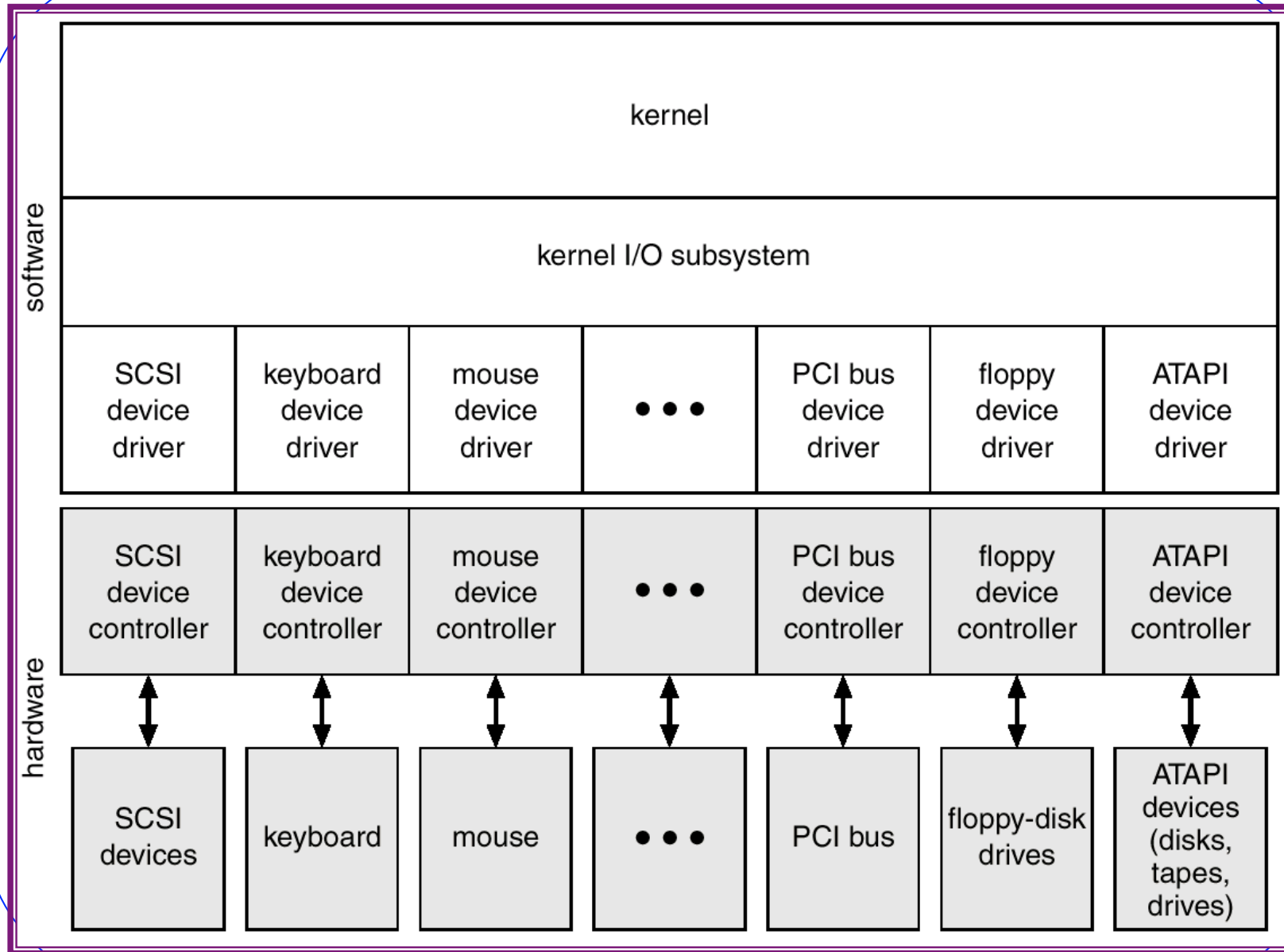


Driver dei Dispositivi di I/O

Software (spesso di terze parti) che accede al controller dei device

- Hanno la vera conoscenza di come far funzionare il dispositivo
- Implementano le funzionalità standardizzate, secondo poche classi (ad es.: carattere/blocchi)
- Vengono eseguiti in modalità kernel
- Per includere un driver, può essere necessario ricompilare o rilinkare il kernel.
- Attualmente si usa un meccanismo di caricamento run-time

Struttura Relativa all' I/O del Kernel





Driver dei Dispositivi di I/O

Struttura di un driver

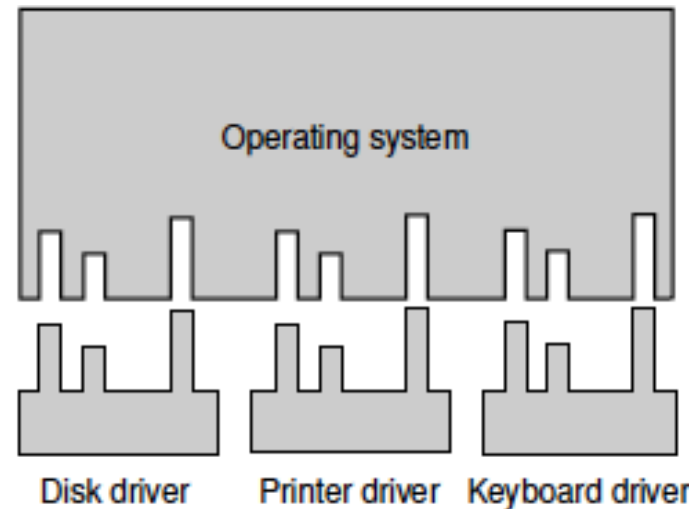
Passi eseguiti dai driver dei dispositivi

1. Controllare i parametri passati
2. Accodare le richieste di una coda di operazioni (soggette a scheduling!)
3. Eseguire le operazioni, accedendo al controller
4. Passare il processo in modo wait (I/O interrupt-driven), o attendere la fine dell'operazione in busy-wait.
5. Controllare lo stato dell'operazione nel controller
6. Restituire il risultato.

Driver dei Dispositivi di I/O

Struttura di un driver

- I driver devono essere rientranti : a metà di una esecuzione, può essere lanciata una nuova esecuzione.
- I driver non possono eseguire system call (sono sotto), ma possono accedere ad alcune funzionalità del kernel (es: allocazione memoria per buffer di I/O)
- Nel caso di dispositivi “hot plug”: gestire l’inserimento/disinserimento a caldo.
- I driver devono avere una interfaccia uniforme
 - Gli sviluppatori dei driver si devono attenere ad una specifica di cosa devono implementare
 - Deve offrire anche un modo di denominazione uniforme, flessibile e generale





Driver dei Dispositivi di I/O

Esempio: Interfaccia dei driver di Linux

```
struct file_operations {
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char *, size_t, loff_t *);
    int (*readdir) (struct file *, void *, filldir_t);
    unsigned int (*poll) (struct file *, struct poll_table_struct *);
    int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, struct dentry *, int datasync);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    ssize_t (*readv) (struct file *, const struct iovec *, unsigned long, loff_t *);
    ssize_t (*writev) (struct file *, const struct iovec *, unsigned long, loff_t *);
    ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t *, int);
    unsigned long (*get_unmapped_area)(struct file *, unsigned long, ...);
};
```



Driver dei Dispositivi di I/O

Esempio: driver di una tastiera

Assunzioni semplificative

- La periferica è un dispositivo di input di tipo carattere.
 - ➔ i registri **comando** e **stato** si riducono ad un unico registro **flag** (flag = 0 equivale a richiesta di operazione e flag = 1 equivale a operazione completata)
- La sola operazione possibile è il trasferimento di un carattere in ingresso.
 - ➔ attraverso il registro di interfaccia **dati**.
- Il driver non esegue controlli su eventuali malfunzionamenti.
- La procedura **leggi_linea** restituisce in uscita il numero dei caratteri effettivamente trasferiti.
- L'operazione è considerata conclusa se giunge un carattere di fine linea o se il buffer è pieno.

(continua ...)



Driver dei Dispositivi di I/O

Esempio: driver di una tastiera

(... continua)

- La lunghezza della linea di caratteri e il carattere da interpretare come fine linea possono essere selezionati attraverso la procedura **inizializza**.
- La mutua esclusione nell'esecuzione delle procedure viene realizzata attraverso il meccanismo di **disabilitazione delle interruzioni**.



Driver dei Dispositivi di I/O

Esempio: driver di una tastiera

```
#define BUFFER_SIZE 10

/* Device Control Block */
typedef struct {
    /* buffer di memorizzazione dei caratteri */
    char    buffer[BUFFER_SIZE];
    /* numero caratteri trasferiti */
    int     count;
    /* carattere di fine linea */
    char    eol;
    /* lunghezza massima di una linea */
    int     lungh_linea;
    /* coda di attesa per processo richiedente */
    PCB     *trasf_avvenuto;
    /* operazione in corso */
    boolean occupato;
} t_DCB;
```



Driver dei Dispositivi di I/O

Esempio: driver di una tastiera

```
/* registri hardware del controller */
typedef struct {
    /* registro di lettura e scrittura dati */
    char    dati;
    /* registro di comando e di stato */
    boolean flag;
} t_registri;

t_DCB          DCB;
t_registri     registri;
```



Driver dei Dispositivi di I/O

Esempio: driver di una tastiera

```
/*      Procedura di richiesta operazione -  
        richiamato dalla system call */  
int leggi_linea(PCB p; char *local_buffer)  
{  
    if (DCB.occupato)  
        return -1;  
    else  
    {  
        DCB.occupato = 1; /* inizializzazione */  
        DCB.count = 0;  
        registri.flag = 0; /*comando iniz. controller */  
        /* attesa del completamento operazione */  
        add p to DCB.trasf_avvenuto;  
        local_buffer = DCB.buffer /*trasferimento dati*/  
        DCB.occupato = 0;  
        return DCB.count;  
    }  
}
```



Driver dei Dispositivi di I/O

Esempio: driver di una tastiera

```
/* Interrupt handler: richiamato dall'interrupt HW */  
void risposta_interruzione()  
{  
    /* acquisizione di un carattere */  
    DCB.count++;  
    DCB.buffer[DCB.count] = registri.dati;  
    if (DCB.count == 80) ||  
        (DCB.buffer[DCB.count] == DCB.eol)  
        /* fine dell'operazione */  
        resume p from DCB.traf_avvenuto;  
    else  
        /* nuovo comando al controller */  
        registri.flag = 0;  
}
```

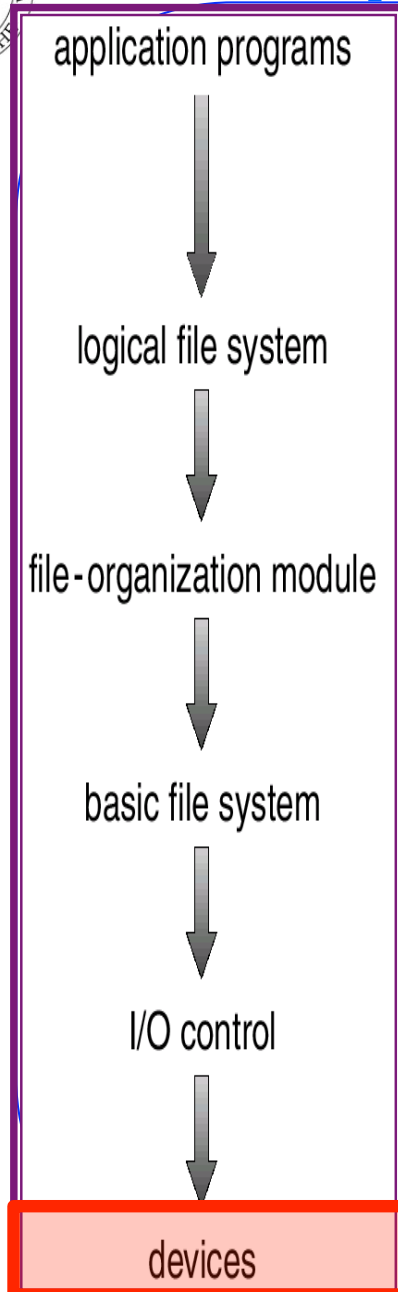



Driver dei Dispositivi di I/O

Esempio: driver di una tastiera

```
/* Procedura di inizializzazione */  
void  inizializza(int lunghezza; char eol)  
{  
    DCB.lungh_linea = lunghezza;  
    DCB.eol = eol;  
    DCB.occupato = 0;  
}
```

File System Stratificato



1. Interazione con il file system mediante **system call**. Il file è individuato dal suo **pathname** o dal **FCB handler**. Le librerie di programma trasformano la richiesta di accesso ad un record nella richiesta di accesso ad un dato **blocco logico**.
2. Reperimento dei file all'interno della gerarchia delle directory. Gestione dei metadati (nei descrittori di file o FCB). Es: tipo di file e dispositivo (**carattere** o **blocco**), file location, protezioni, **numero di blocco logico**.
3. Per i dispositivi di tipo blocco; assegnazione dei blocchi liberi del disco; traduzione numeri di blocco logici in **numeri di blocco fisici**.
4. **Caching, buffering, scheduling**. Per i dispositivi di tipo blocco, traduzione dei numeri di blocco fisici in indirizzi del supporto di memorizzazione (**<superficie, cilindro, settore>**). Comunicazione con i driver per leggere e scrivere blocchi fisici o caratteri.
5. **Driver dei dispositivi**. Interazione con i controller (lettura e scrittura nei registri del controller).
6. **Controller**. Interazione con i dispositivi



Hardware dei Dispositivi di I/O

- Esiste una incredibile varietà di dispositivi di I/O
- Concetti comuni
 - Porta
 - Bus
 - Controller
- I dispositivi hanno indirizzi usati da
 - istruzioni dirette di I/O
 - memory-mapped I/O



Indirizzi delle Porte dei Dispositivi di I/O nei PC (elenco parziale)

I/O address range (hexadecimal)	device
000-00F	DMA controller
020-021	interrupt controller
040-043	timer
200-20F	game controller
2F8-2FF	serial port (secondary)
320-32F	hard-disk controller
378-37F	parallel port
3D0-3DF	graphics controller
3F0-3F7	diskette-drive controller
3F8-3FF	serial port (primary)

Caratteristiche dei Dispositivi di I/O

aspect	variation	example
data-transfer mode	character block	terminal disk
access method	sequential random	modem CD-ROM
transfer schedule	synchronous asynchronous	tape keyboard
sharing	dedicated sharable	tape keyboard
device speed	latency seek time transfer rate delay between operations	
I/O direction	read only write only read-write	CD-ROM graphics controller disk



Dispositivi a Blocchi e a Caratteri

■ Dispositivi a Blocchi

I dispositivi a blocchi comprendono i dischi

- Sono a disposizione istruzioni quali `read`, `write`, `seek`.
- Le applicazioni comunicano attraverso l'interfaccia del **File System** oppure attraverso l'**I/O a basso livello**.
- È possibile l'accesso memory-mapped

■ Dispositivi a Caratteri

I dispositivi a carattere comprendono tastiere, mouse, porte seriali

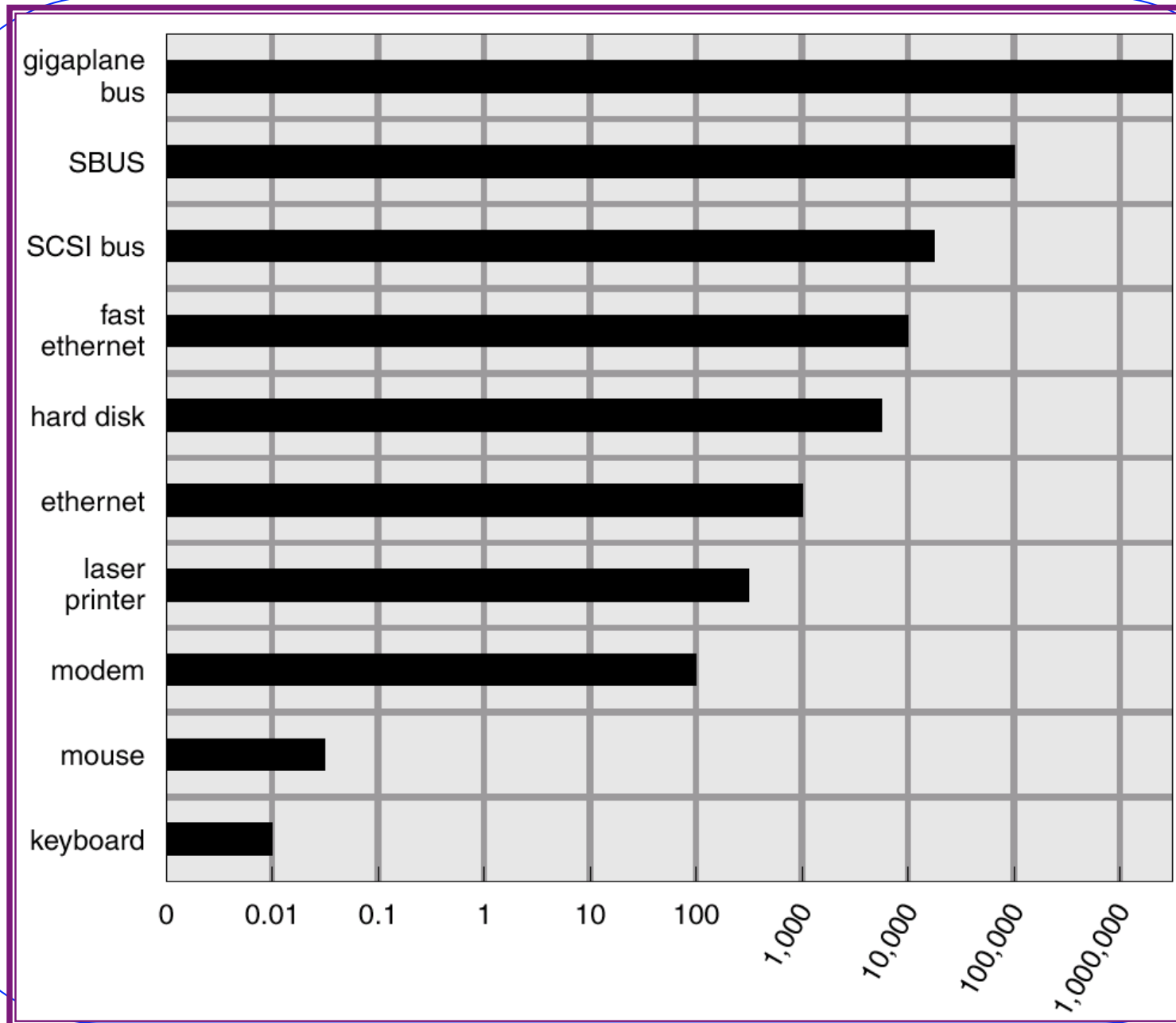
- Sono a disposizione istruzioni quali `get`, `put`.
- Esistono librerie che permettono operazioni di editing (quali l'accesso riga per riga o il `backspace`)

■ Dispositivi di Rete

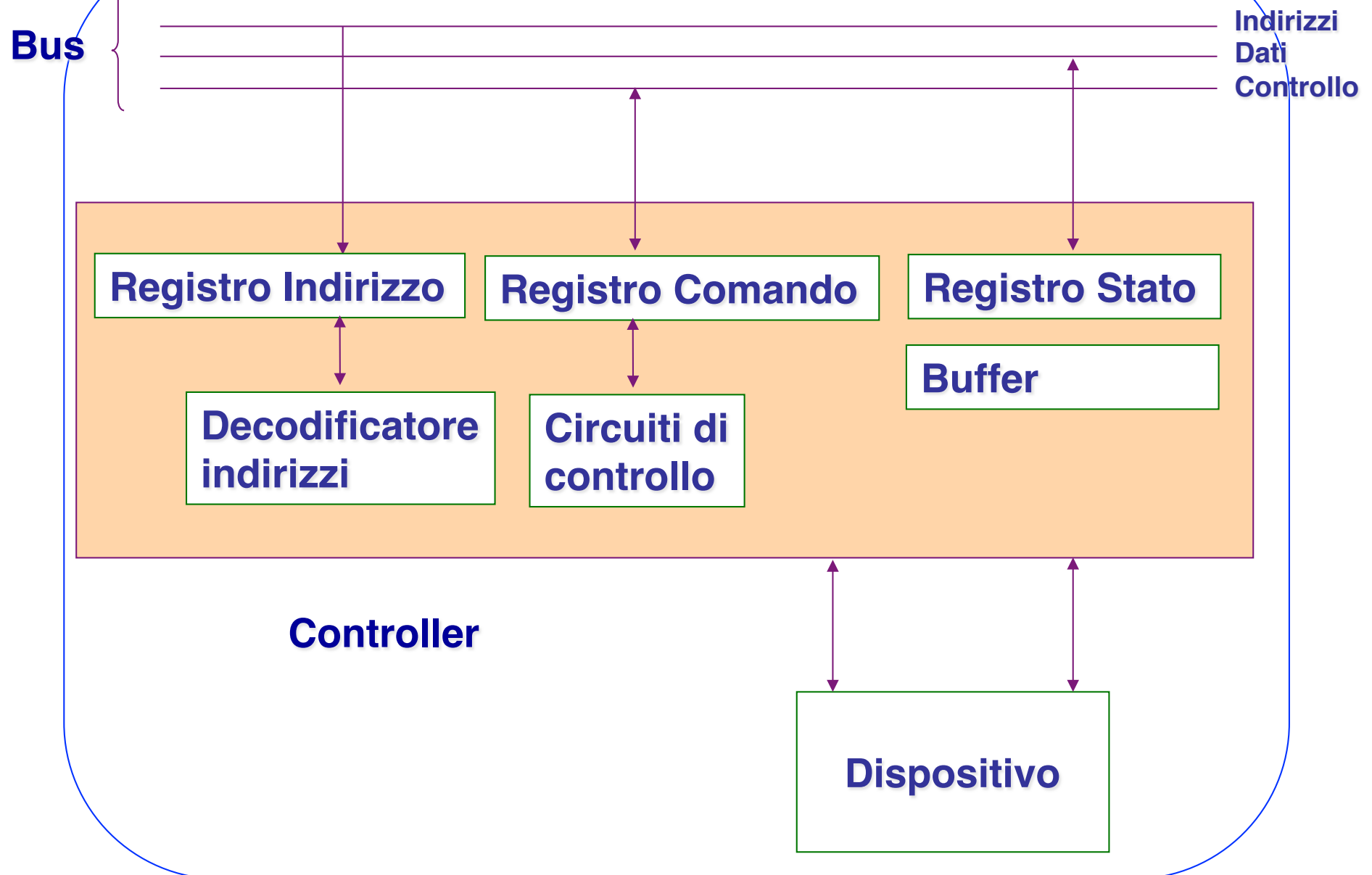
I dispositivi di rete sono profondamente diversi dagli altri, tanto da avere una interfaccia propria.

- Ad esempio le applicazioni comunicano attraverso l'interfaccia delle **socket**.

Velocità di Trasferimento di alcuni Dispositivi



Controller dei Dispositivi di I/O





Controller dei Dispositivi di I/O

Ogni dispositivo e' collegato al bus di sistema tramite un'interfaccia dove risiedono alcuni registri (di controllo, per permettere la sincronizzazione CPU-Dispositivo, buffer per contenere i dati in ingresso/uscita). Questa interfaccia e' chiamata **controller**.

La CPU colloquia con il controller scrivendo/leggendolo i suoi registri:

- utilizzando apposite istruzioni (**IN / OUT**) che avranno come parametro un "indirizzo" che identifica un particolare controller
- utilizzando normali istruzioni **LOAD/STORE** a indirizzi associati ai registri del controller: questa tecnica si chiama Memory Mapped I/O



Controller dei Dispositivi di I/O

Nelle architetture più semplici si trovano normalmente tre tipi di schemi per eseguire le azioni di I/O

- I/O programmato con Busy-Waiting
- I/O guidato dalle interruzioni (Interrupt)
- I/O con accesso diretto alla memoria (DMA)



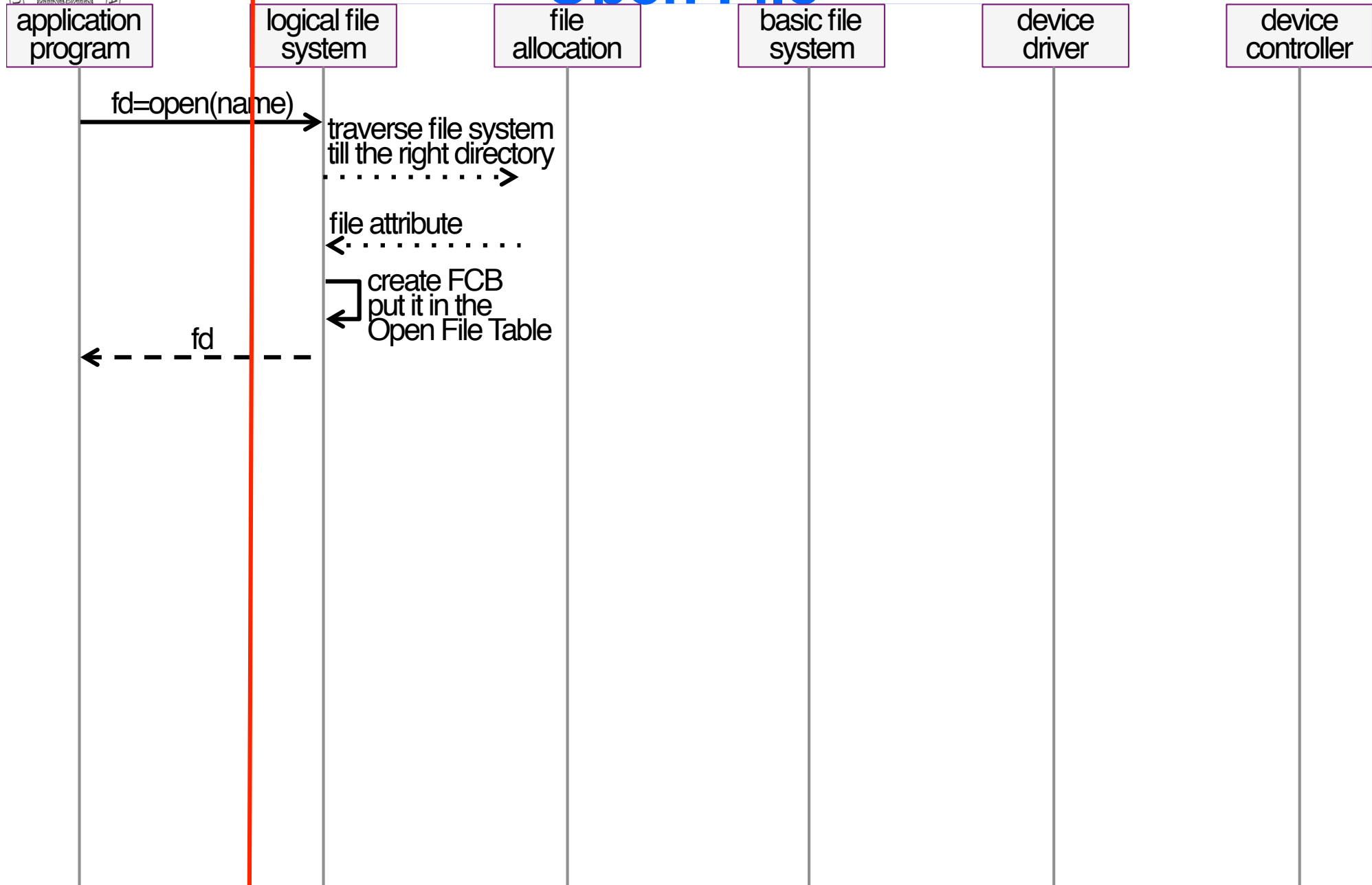
Gestione dell' I/O

- Gestione dell' Unità a Disco
 - Struttura del disco
 - Scheduling del disco
 - Gestione del disco
 - Strutture RAID
- Gestione dei dispositivi di I/O
 - Driver dei dispositivi di I/O
 - Hardware e Controller dei dispositivi di I/O
- Ciclo di vita di una richiesta di I/O
- Gestione dell' I/O in Linux
 - Struttura
 - Tipi di dispositivi
 - Scheduling del disco

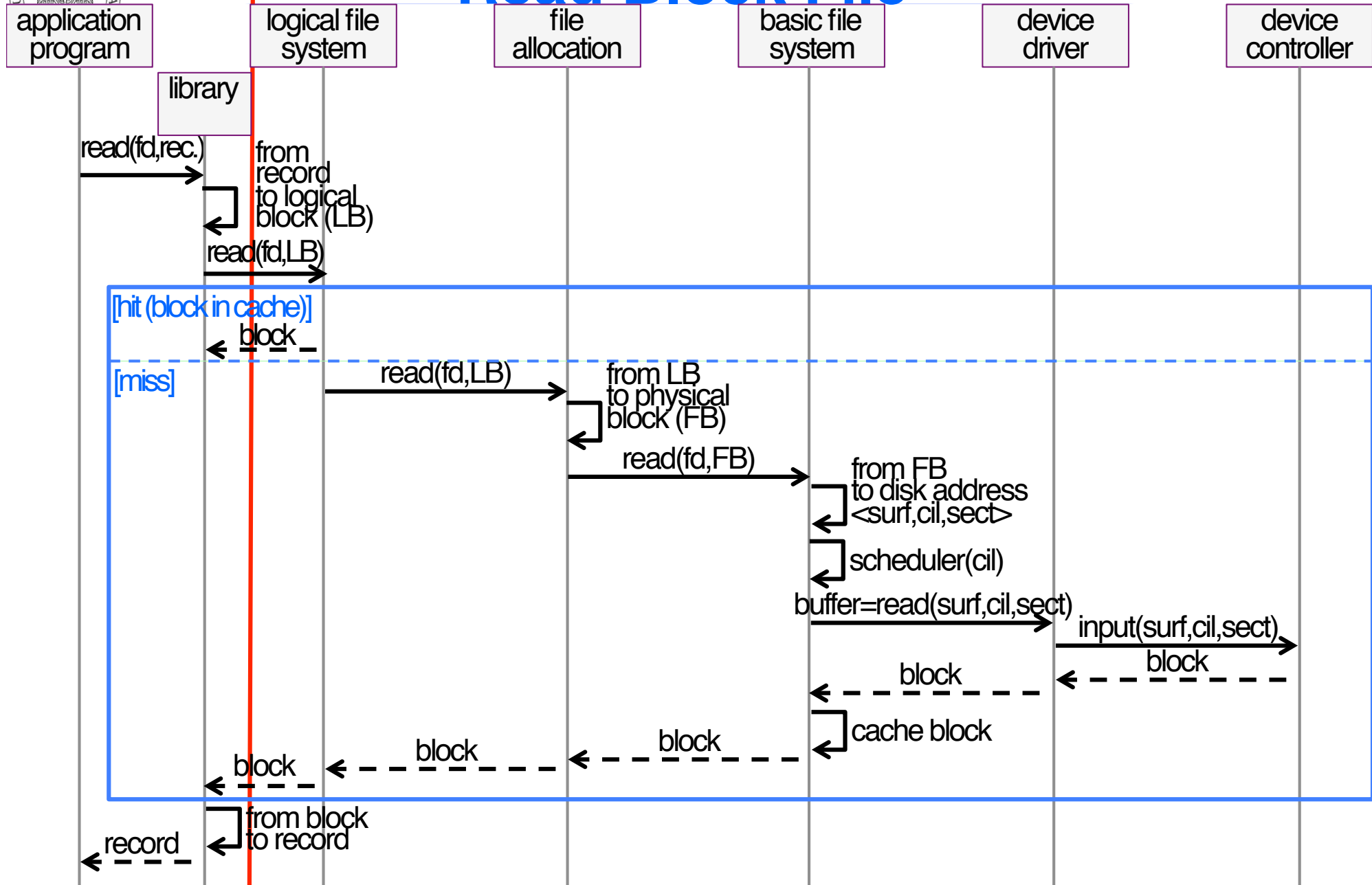


user mode | kernel mode

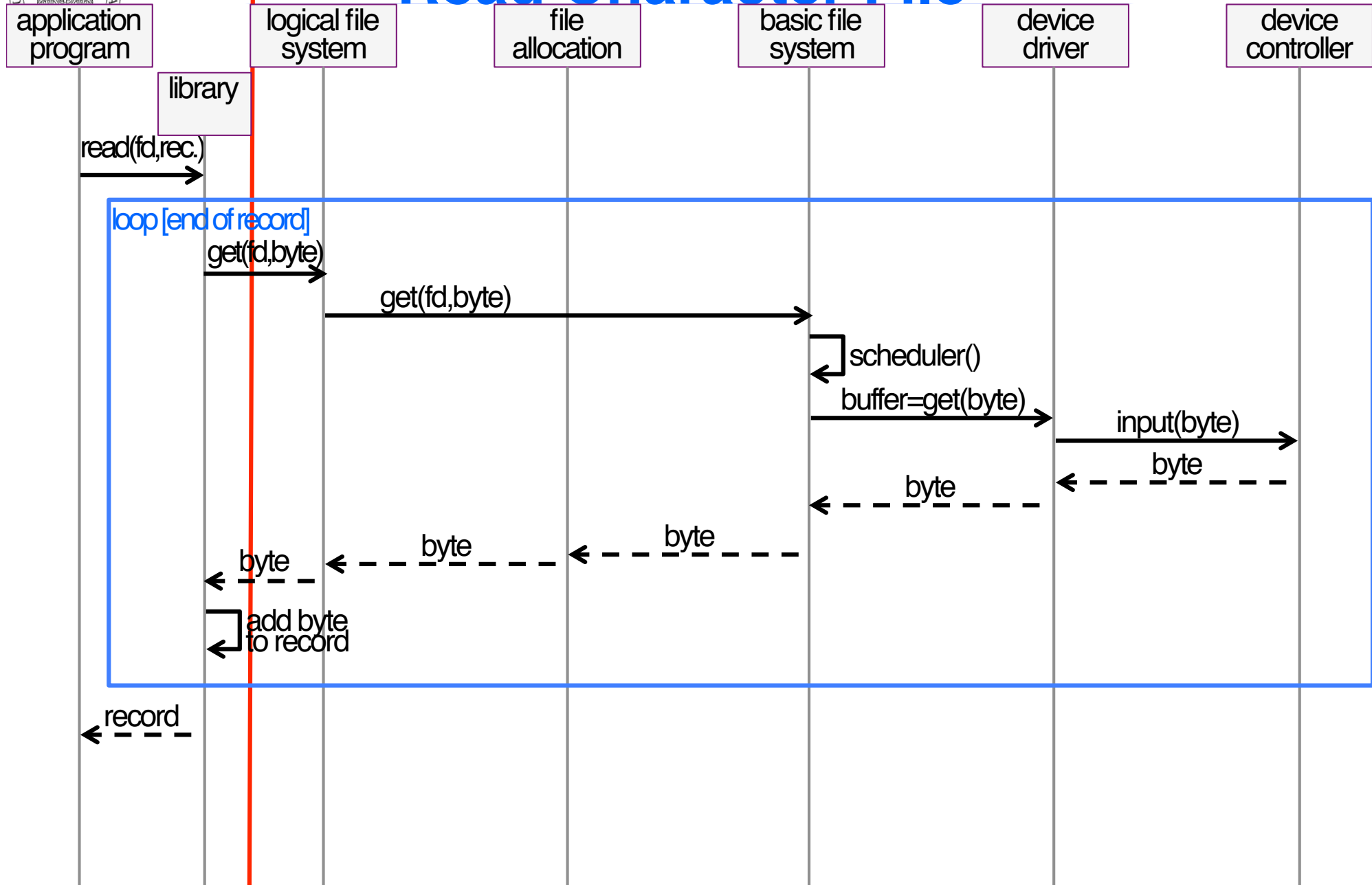
Open File



Read Block File



Read Character File





Gestione dell' I/O

- Gestione dell' Unità a Disco
 - Struttura del disco
 - Scheduling del disco
 - Gestione del disco
 - Strutture RAID
- Gestione dei dispositivi di I/O
 - Driver dei dispositivi di I/O
 - Hardware e Controller dei dispositivi di I/O
- Ciclo di vita di una richiesta di I/O
- Gestione dell' I/O in Linux
 - Struttura
 - Tipi di dispositivi
 - Scheduling del disco

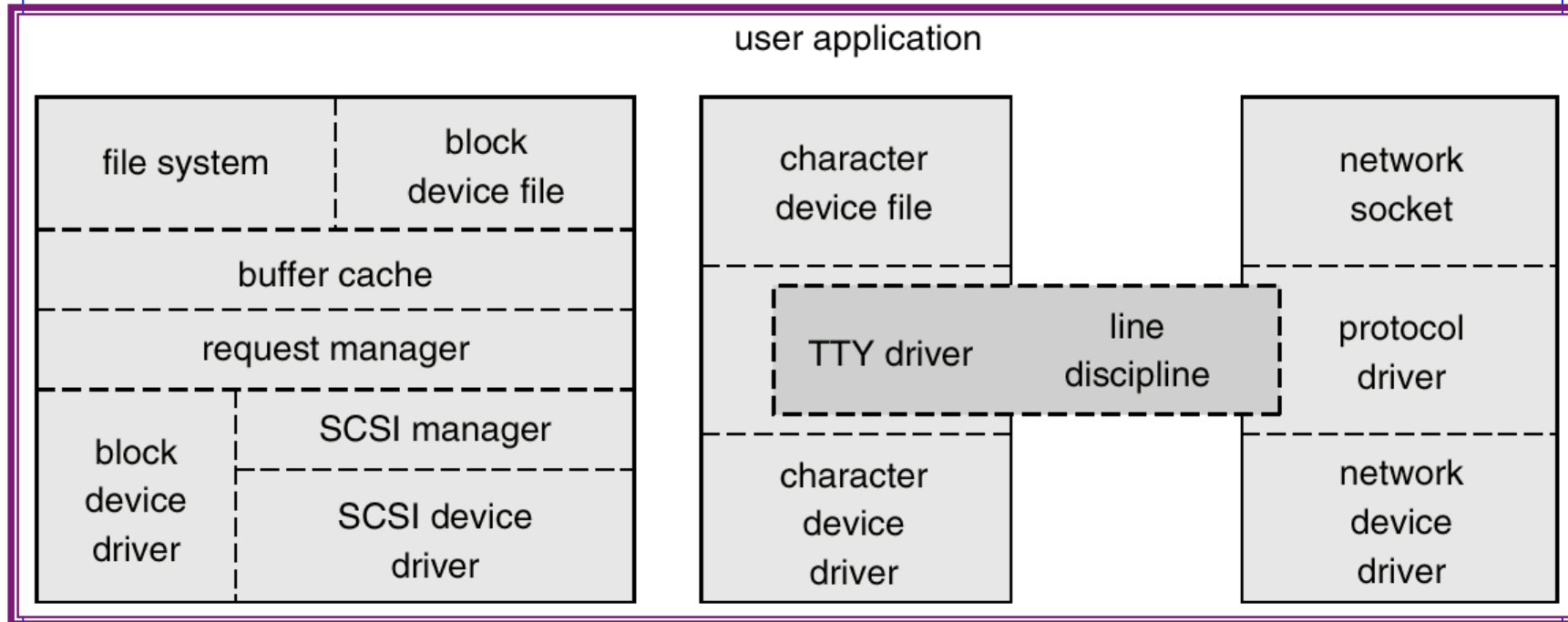


Gestione dell' I/O in Linux

- Linux classifica tutti i dispositivi in tre categorie:
 - *Dispositivi a blocchi*, permettono l' accesso diretto a blocchi di dati di dimensione fissa.
 - *Dispositivi a caratteri*, comprendono gran parte dei dispositivi, non sono in grado di fornire le funzionalità dei file ordinari.
 - *Dispositivi di rete*, sono interfacciati con il sottosistema di gestione delle reti del kernel.

Gestione dell' I/O in Linux

Struttura





Gestione dell' I/O in Linux

Dispositivi a Blocchi

- È la principale interfaccia a tutti i dischi del sistema.
- Linux accede ai dischi attraverso due cache:
 - I dati sono memorizzati nella page cache, che è unita alla memoria virtuale del sistema;
 - I metadati sono memorizzati nella buffer cache, una cache separata indicizzata dal blocco fisico del disco.
- Il *block buffer* cache serve principalmente a due cose:
 - come pool di buffer per I/O attivo,
 - come cache per l' I/O completato.
- Il *request manager* gestisce la scrittura e la lettura del buffer con dati provenienti da/diretti verso un driver di un dispositivo a blocchi.



Gestione dell' I/O in Linux

Dispositivi a Caratteri

- Un driver di un dispositivo a caratteri deve avere un insieme di funzioni che realizzano le varie operazioni sui file.
- Il kernel non esegue quasi nessuna operazione alla richiesta di leggere o scrivere su un dispositivo a caratteri, semplicemente passa la richiesta al dispositivo.
- La sola eccezione è rappresentata dai terminali, nei confronti dei quali il kernel mantiene una interfaccia standard.



Gestione dell' I/O in Linux

Dispositivi di Rete

- I dispositivi di rete in Linux permettono di gestire:
 - Il protocollo standard di Internet (TCP/IP) per la comunicazione tra macchine Unix,
 - Protocolli per macchine con SO non Unix, come Appletalk e IPX.
- È realizzato con tre strati di software:
 - L' interfaccia delle socket (permette la gestione della rete come se fosse un file)
 - I driver dei protocolli
 - I driver dei dispositivi di rete



Gestione dell' I/O in Linux

Scheduling del disco

- **Linus Elevator scheduler** (fino al kernel 2.4)
 - Utilizza una coda sola sia per le richieste di lettura che di scrittura.
 - Le richieste sono ordinate per numero di blocco.
 - Si muove in una sola direzione
 - Simile a SCAN

Linus Torvalds





Gestione dell' I/O in Linux

Scheduling del disco

■ Deadline scheduler (dal kernel 2.6)

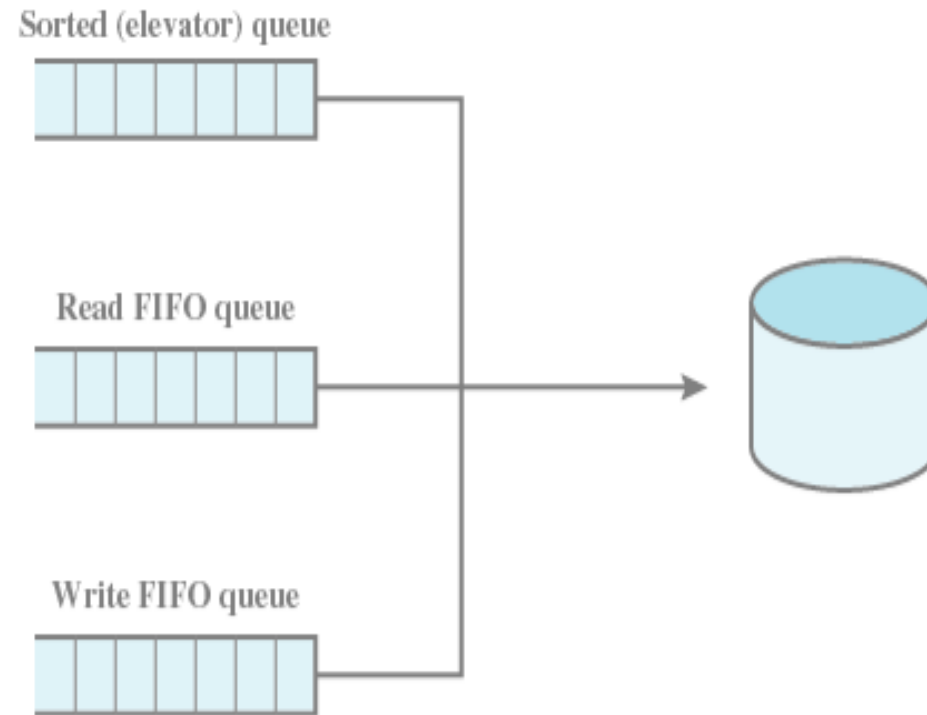
- Ha sostituito *Linus Elevator scheduler*
- Utilizza tre code
 - > **Incoming request** – coda dove vengono inserite tutte le richieste ordinate per numero di blocco.
 - > **Read request** – coda gestita FIFO dove vengono inserite tutte le richieste di lettura presenti nella *incoming request*.
 - > **Write request** – coda gestita FIFO dove vengono inserite tutte le richieste di scrittura presenti nella *incoming request*.
- Le richieste sono rimosse dalle code *read request* e *write request* se sono state servite dall' *elevator scheduler*.
- Ogni richiesta ha una scadenza (*expiration time*), alla scadenza della quale se non è stata servita il controllo passa al *FIFO scheduler* (evita la starvation).



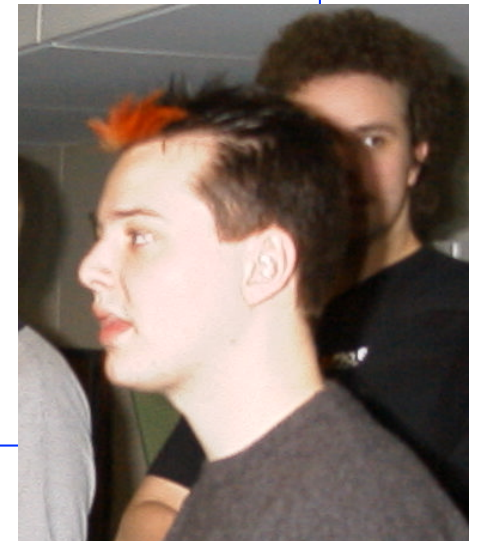
Gestione dell' I/O in Linux

Scheduling del disco

■ Deadline scheduler



Jens Axboe





Gestione dell' I/O in Linux

Scheduling del disco

- **Anticipatory I/O scheduler** (fino al kernel 2.6.33)
 - I programmi spesso utilizzano settori consecutivi.
 - Dopo aver soddisfatto una richiesta aspetta per un breve periodo (fino a 6ms) per vedere se arriva una richiesta di un settore vicino.

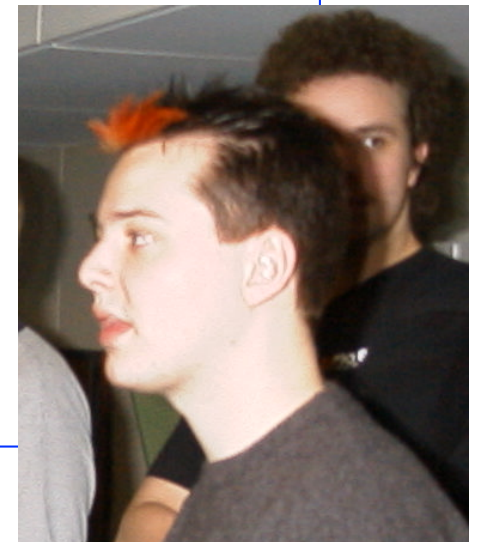


Gestione dell' I/O in Linux

Scheduling del disco

- **CFQ – Completely Fair Queueing** (dal kernel 2.6.33)
 - Sostituisce *Anticipatory Scheduler*
 - Richieste di I/O sincrone
 - > Mantiene una coda FIFO di richieste per ogni processo
 - Richieste di I/O asincrone
 - > Mantiene un coda FIFO di richieste per priorità
 - Round Robin tra le diverse code
 - > Il quanto di tempo è in funzione della priorità del processo e di altri parametri (è abbastanza complesso)

Jens Axboe





Gestione dell' I/O in Linux

Scheduling del disco

■ Noop

- FCFS.
- Richiede il minor overhead al kernel



Gestione dell' I/O in Linux

Scheduling del disco

■ Scelta dello scheduler

- `cat /sys/block/<device>/queue/scheduler`
noop deadline [cfq]

■ Elenca tutti gli scheduler che posso utilizzare per quel device, quello tra parentesi quadre è quello attualmente in uso